



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 04

NOMBRE COMPLETO: Hernández Domínguez Luis Carlos

N° de Cuenta: 320182668

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 04

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 21/09/2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1. Ejercicios

Ejercicio 1: Terminar la grúa con:

- cuerpo (prisma rectangular
- base (pirámide triangular
- 4 llantas (4 cilindros) con teclado se pueden girar las 4 llantas por separado

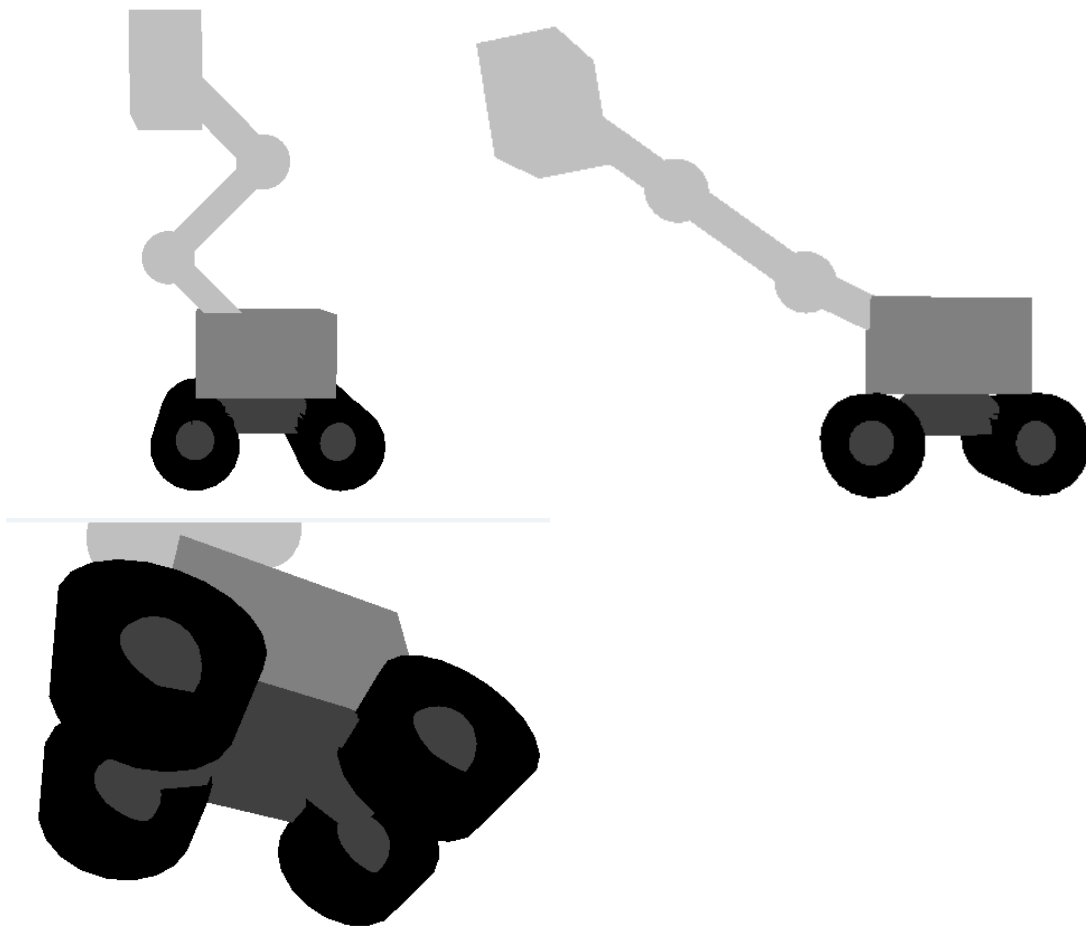
Bloques de código generado

```
463     model = modelaux = modelaux2;
464
465     //Base
466     model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
467     modelaux = model;
468     model = glm::scale(model, glm::vec3(5.0f, 2.5f, 3.0f));
469     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
470     color = glm::vec3(0.25f, 0.25f, 0.25f);
471     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
472     meshList[4] -> RenderMeshGeometry(); //dibuja cubo y pirámide triangular
473
474     //articulación 5
475     model = modelaux;
476     model = glm::translate(model, glm::vec3(-2.5f, -1.5f, -1.5f));
477     model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
478     model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, 1.0f));
479     color = glm::vec3(0.25f, 0.25f, 0.25f);
480     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
481     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
482     sp.render();
483
484     //Rueda 1
485     model = glm::scale(model, glm::vec3(2.2f, 2.2f, 2.2f));
486     model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
487     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
488     color = glm::vec3(0.0f, 0.0f, 0.0f);
489     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
490     meshList[2] -> RenderMeshGeometry(); //dibuja cubo y pirámide triangular
```

```
492     //articulación 6
493     model = modelaux;
494     model = glm::translate(model, glm::vec3(-2.5f, -1.5f, 1.5f));
495     model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
496     model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f, 0.0f, 1.0f));
497     color = glm::vec3(0.25f, 0.25f, 0.25f);
498     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
499     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
500     sp.render();
501
502     //Rueda 2
503     model = glm::scale(model, glm::vec3(2.2f, 2.2f, 2.2f));
504     model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
505     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
506     color = glm::vec3(0.0f, 0.0f, 0.0f);
507     glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
508     meshList[2] -> RenderMeshGeometry(); //dibuja cubo y pirámide triangular
509
510     //articulación 7
511     model = modelaux;
512     model = glm::translate(model, glm::vec3(2.5f, -1.5f, -1.5f));
513     model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
514     model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(0.0f, 0.0f, 1.0f));
515     color = glm::vec3(0.25f, 0.25f, 0.25f);
516     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
517     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
518     sp.render();
```

```
520     //Rueda 3
521     model = glm::scale(model, glm::vec3(2.2f, 2.2f, 2.2f));
522     model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
523     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
524     color = glm::vec3(0.0f, 0.0f, 0.0f);
525     glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
526     meshList[2] -> RenderMeshGeometry(); //dibuja cubo y pirámide triangular
527
528     //articulación 8
529     model = modelaux;
530     model = glm::translate(model, glm::vec3(2.5f, -1.5f, 1.5f));
531     model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
532     model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 0.0f, 1.0f));
533     color = glm::vec3(0.25f, 0.25f, 0.25f);
534     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
535     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
536     sp.render();
537
538     //Rueda 4
539     model = glm::scale(model, glm::vec3(2.2f, 2.2f, 2.2f));
540     model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
541     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
542     color = glm::vec3(0.0f, 0.0f, 0.0f);
543     glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
544     meshList[2] -> RenderMeshGeometry(); //dibuja cubo y pirámide triangular
```

Ejecución



Para resolver este problema continuando a partir del ejercicio de clase fue necesaria una segunda variable auxiliar para poder regresar al cuerpo de la grúa y a partir de allí ir jerarquizando la base y las cuatro ruedas con sus respectivas articulaciones. De igual forma, fue necesario agregar más articulaciones en `window.h` y `window.cpp`.

Ejercicio 2: Crear un animal robot 3d

- instanciando cubos, pirámides, cilindros, conos, esferas:

- 4 patas articuladas en 2 partes (con teclado se puede mover las dos articulaciones de cada pata)

- cola articulada o 2 orejas articuladas. (con teclado se puede mover la cola o cada oreja independiente)

Bloques de código generado

```
559 //Luerpql
560 model = glm::mat4(1.0);
561 model = glm::translate(model, glm::vec3(0.0f, 1.0f, 0.0f));
562 modelaux2 = modelaux = model;
563 model = glm::scale(model, glm::vec3(3.6f, 3.0f, 2.4f));
564 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
565 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
566 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
567 color = glm::vec3(0.75f, 0.75f, 0.75f);
568 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
569 meshList[0]->RenderMesh();
570
571 //Data delantera derecha
572 //articulacion 1
573 model = modelaux;
574 model = glm::translate(model, glm::vec3(0.3f, 0.5f, 1.4f));
575 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, -1.0f));
576 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
577 color = glm::vec3(0.75f, 0.75f, 0.75f);
578 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
579 sp.render();
580
581 model = glm::translate(model, glm::vec3(-0.1f, -0.45f, 0.0f));
582 model = glm::rotate(model, glm::radians(350.0f), glm::vec3(0.0f, 0.0f, 1.0f));
583 modelaux = model;
584 model = glm::scale(model, glm::vec3(1.8f, 2.6f, 1.3f));
585 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
586 color = glm::vec3(0.25f, 0.25f, 0.25f);
587 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
588 meshList[0]->RenderMesh();
```

```
590 //Articulacion 2
591 model = modelaux;
592 model = glm::translate(model, glm::vec3(0.0f, -1.2f, 0.0f));
593 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, -1.0f));
594 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
595 color = glm::vec3(0.75f, 0.75f, 0.75f);
596 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
597 sp.render();
598
599 model = glm::translate(model, glm::vec3(0.7f, -1.35f, 0.0f));
600 model = glm::rotate(model, glm::radians(30.0f), glm::vec3(0.0f, 0.0f, 1.0f));
601 modelaux = model;
602 model = glm::scale(model, glm::vec3(1.1f, 2.4f, 1.2f));
603 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
604 color = glm::vec3(0.25f, 0.25f, 0.25f);
605 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
606 meshList[0]->RenderMesh();
607
608 model = modelaux;
609 model = glm::rotate(model, glm::radians(340.0f), glm::vec3(0.0f, 0.0f, 1.0f));
610 model = glm::translate(model, glm::vec3(0.5f, -1.08f, 0.0f));
611 modelaux = model;
612 model = glm::scale(model, glm::vec3(2.0f, 0.5f, 1.0f));
613 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
614 color = glm::vec3(0.25f, 0.25f, 0.25f);
615 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
616 meshList[0]->RenderMesh();
```

```
618 //garias
619 model = glm::translate(model, glm::vec3(0.5f, -0.25f, -0.4f));
620 modelaux = model;
621 model = glm::scale(model, glm::vec3(0.5f, 0.2f, 0.2f));
622 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
623 color = glm::vec3(1.0f, 1.0f, 1.0f);
624 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
625 meshList[0]->RenderMesh();
626
627 model = modelaux;
628 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.4f));
629 modelaux = model;
630 model = glm::scale(model, glm::vec3(0.5f, 0.2f, 0.2f));
631 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
632 color = glm::vec3(1.0f, 1.0f, 1.0f);
633 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
634 meshList[0]->RenderMesh();
635
636 model = modelaux;
637 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.4f));
638 modelaux = model;
639 model = glm::scale(model, glm::vec3(0.5f, 0.2f, 0.2f));
640 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
641 color = glm::vec3(1.0f, 1.0f, 1.0f);
642 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
643 meshList[0]->RenderMesh();
```

```

839 //segunda seccion del torso
840 model = modelaux;
841 model = glm::translate(model, glm::vec3(-2.7f, 0.0f, 0.0f));
842 modelaux2 = modelaux = model;
843 model = glm::scale(model, glm::vec3(2.5f, 2.6f, 2.0f));
844 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
845 color = glm::vec3(0.25f, 0.25f, 0.25f);
846 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
847 meshList[0] -> RenderMesh();
848
849 //Data trasera derecha
850 //articulacion 5
851 model = modelaux;
852 model = glm::translate(model, glm::vec3(-1.6f, 0.5f, 1.2f));
853 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, -1.0f));
854 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
855 color = glm::vec3(0.75f, 0.75f, 0.75f);
856 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
857 sp.render();
858
859 model = glm::translate(model, glm::vec3(0.0f, -1.0f, 0.0f));
860 modelaux = model;
861 model = glm::scale(model, glm::vec3(1.3f, 2.5f, 1.2f));
862 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
863 color = glm::vec3(0.25f, 0.25f, 0.25f);
864 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
865 meshList[0] -> RenderMesh();

```

```

867 //articulacion 6
868 model = modelaux;
869 model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
870 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f, 0.0f, -1.0f));
871 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
872 color = glm::vec3(0.75f, 0.75f, 0.75f);
873 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
874 sp.render();
875
876 model = glm::rotate(model, glm::radians(25.0f), glm::vec3(0.0f, 0.0f, 1.0f));
877 model = glm::translate(model, glm::vec3(0.0f, -1.0f, 0.0f));
878 modelaux = model;
879 model = glm::scale(model, glm::vec3(1.1f, 2.0f, 1.0f));
880 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
881 color = glm::vec3(0.25f, 0.25f, 0.25f);
882 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
883 meshList[0] -> RenderMesh();
884
885 model = modelaux;
886 model = glm::rotate(model, glm::radians(335.0f), glm::vec3(0.0f, 0.0f, 1.0f));
887 model = glm::translate(model, glm::vec3(0.5f, -1.0f, 0.0f));
888 modelaux = model;
889 model = glm::scale(model, glm::vec3(2.0f, 0.5f, 1.0f));
890 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
891 color = glm::vec3(0.25f, 0.25f, 0.25f);
892 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
893 meshList[0] -> RenderMesh();

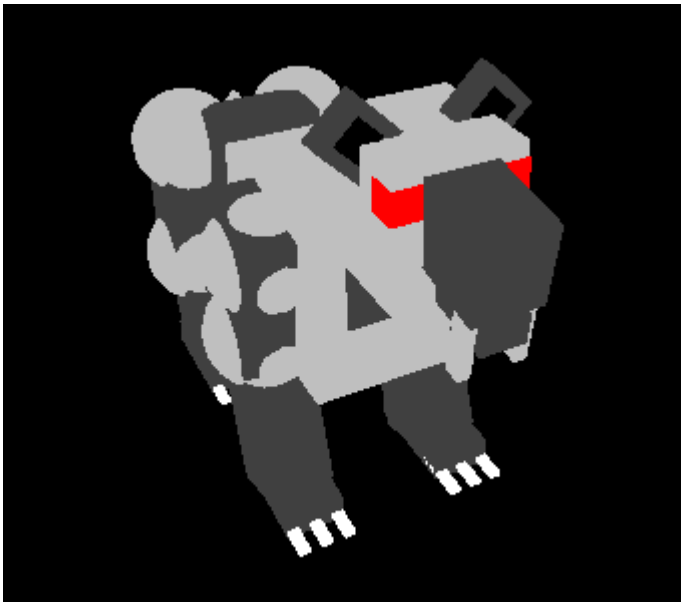
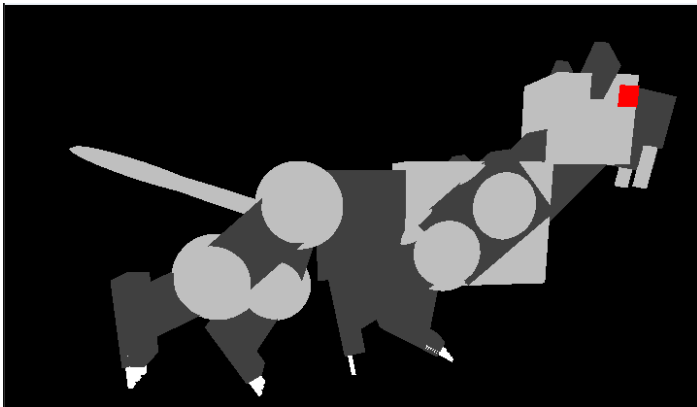
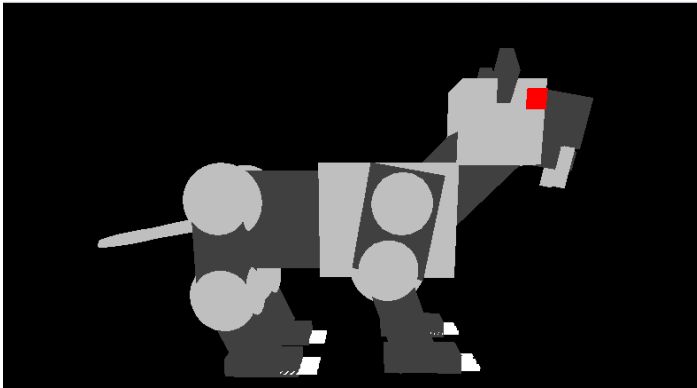
```

```

997 //articulación 9
998 model = modelaux = modelaux2;
999 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion9()), glm::vec3(1.0f, 0.0f, 0.0f));
1000 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1001 color = glm::vec3(1.0f, 1.0f, 1.0f);
1002 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1003 sp.render();
1004
1005
1006 model = glm::translate(model, glm::vec3(-2.75f, 0.0f, 0.4f));
1007 model = glm::rotate(model, glm::radians(20.0f), glm::vec3(0.0f, 1.0f, 0.0f));
1008 modelaux = model;
1009 model = glm::scale(model, glm::vec3(3.0f, 0.2f, 0.2f));
1010 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1011 color = glm::vec3(0.75f, 0.75f, 0.75f);
1012 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1013 meshList[2] -> RenderMeshGeometry();
1014
1015
1016 model = modelaux;
1017 model = glm::translate(model, glm::vec3(-2.65f, 0.0f, 0.0f));
1018 //model = glm::rotate(model, glm::radians(30.0f), glm::vec3(0.0f, 1.0f, 0.0f));
1019 model = glm::scale(model, glm::vec3(1.5f, 0.2f, 0.2f));
1020 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1021 color = glm::vec3(0.75f, 0.75f, 0.75f);
1022 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1023 meshList[2] -> RenderMeshGeometry();
1024

```

Ejecución



Para resolver este problema de la misma manera que con la grúa fue necesario definir el nodo padre de toda la estructura que en este caso fue el cuerpo del animal robot en cuestión. E ir jerarquizando y añadiendo las articulaciones necesarias para el modelo.

2. Problemas presentados.

No se presentó ningún problema a la hora de ejecutar código.

3.- Conclusión:

- a. Esta práctica fue bastante interesante porque el modelo jerárquico facilita en algunos aspectos el modelado realizado en la práctica previa con el pyraminx. De igual forma, el intentar crear un modelo con solo usar cuerpos geométricos y jerarquizarlos para mover las extremidades correctamente fue algo bastante interesante y nuevo.
- b. Los conceptos y explicaciones dadas en clase fueron correctas y suficientes para la resolución de esta práctica.
- c. En conclusión, esta práctica me pareció tener un nivel de dificultad adecuado y hace un buen trabajo al introducir el modelo jerárquico.